

SOLID Design Principles

Part 1: SRP, OCP, and LSP — Architecture at Scale

1. Why Do We Need Design Principles?

As applications grow from tens of classes to thousands, managing code complexity becomes a monumental task. Without a proper architecture, classes become tightly coupled. A change in one module can trigger disastrous cascading bugs in another.

Introduced by Robert C. Martin (Uncle Bob) in 2000, the **SOLID** principles act as guidelines to prevent messy, tangled code architectures. Implementing these principles ensures your software remains:

- **Maintainable:** Easy to add new features without breaking existing logic.
- **Readable:** Clear and intuitive for new engineers joining the project.
- **Debuggable:** Issues can be isolated and resolved quickly.

2. Single Responsibility Principle (SRP)

Definition: *"A class should have only one reason to change."*

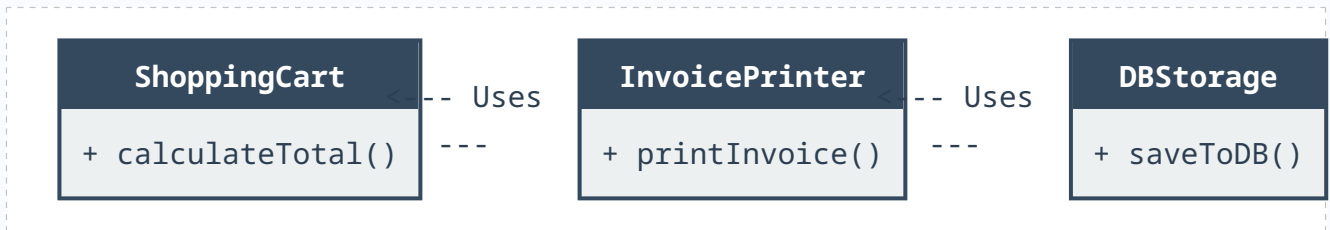
In simple terms, a class should only be responsible for one specific function or part of the software. If a class takes on multiple responsibilities, it becomes highly susceptible to bugs because changing one responsibility could unintentionally break another.

The Anti-Pattern: The "God" Class

Imagine a ShoppingCart class that calculates the total price, prints the invoice, and saves the order to a database. If the database team switches from SQL to MongoDB, you must modify the ShoppingCart class. If the billing team changes the invoice format, you must modify the Shopping Cart class. This violates SRP.

The Solution: Delegation via Composition

Instead of one massive class, we break it down into multiple smaller classes, each handling exactly one job. We then use *Composition* (HAS-A relationship) to link them.



Pro-Tip: Responsibility vs. Methods

SRP does *not* mean a class can only have one method. A DBStorage class might have methods like `connect()`, `query()`, and `disconnect()`. This is perfectly fine because all these methods serve a **single responsibility**: Database interaction.

3. Open/Closed Principle (OCP)

Definition: "Software entities (classes, modules, functions) should be open for extension, but closed for modification."

This means you should be able to add new functionality to a system (Open for Extension) *without* altering the existing, tested code (Closed for Modification).

The Problem

Let's say our DBStorage class currently saves to a SQL database. A new requirement comes in: we need to support saving to MongoDB and standard Files. The naive approach is to open the DBStorage class and add `saveToMongo()` and `saveToFile()`. This directly violates OCP, as modifying existing code risks introducing new bugs into the stable SQL logic.

The Solution: Abstraction & Polymorphism

To adhere to OCP, we use Interfaces (Abstract Classes). We define an abstract base class, and let specific implementations extend it. The client code only interacts with the abstraction.

```
// 1. The Abstraction (Closed for modification)
class DBPersistence {
public:
    virtual void save() = 0; // Pure virtual function
};

// 2. The Extensions (Open for extension)
class SqlPersistence : public DBPersistence {
public:
    void save() override { /* SQL Logic */ }
};

class MongoPersistence : public DBPersistence {
public:
    void save() override { /* Mongo Logic */ }
};
```

Now, if we need to add a `CassandraPersistence`, we simply create a new class. The existing code remains completely untouched!

4. Liskov Substitution Principle (LSP)

Definition: *"Subclasses should be substitutable for their base classes without altering the correctness of the program."*

If class **B** is a child of class **A**, you should be able to pass **B** into any method that expects **A**, and the program should not crash or behave unpredictably. In essence, a child class must *expand* on the parent's capabilities, never *restrict* them.

The Problem: Breaking the Hierarchy

Imagine a base class `Account` with methods `deposit()` and `withdraw()`. `SavingsAccount` and `CurrentAccount` inherit from it beautifully.

Then, the bank introduces a `FixedDepositAccount`. You inherit it from `Account`. However, fixed deposits do not allow withdrawals! To handle this, you override `withdraw()` in `FixedDepositAccount` to throw an Exception.

This breaks LSP. A client iterating through a list of `Account` objects expects to be able to call `withdraw()` safely. When it hits the `Fixed Deposit` object, the program crashes unexpectedly.

The False Fix (Type Checking)

A common developer mistake is adding `if (type == FixedDeposit) { continue; }` inside the client code. This is a massive red flag. Not only does it couple the client to specific child classes, but it also directly violates the Open/Closed Principle!

The Solution: Re-architecting the Interfaces

The core issue is that a Fixed Deposit is *not* logically substitutable for a fully functional Account. The hierarchy is wrong. We must split the interfaces:

- Create a base `DepositOnlyAccount` with a `deposit()` method. Let `FixedDepositAccount` inherit from this.
- Create a `WithdrawableAccount` (which inherits from `DepositOnlyAccount`) and adds the `withdraw()` method. Let `SavingsAccount` and `CurrentAccount` inherit from this.

Now, the client can maintain two separate lists and process them safely. No exceptions are thrown, and LSP is preserved!